vdot.vv 基础测试

本文记录香山 RTL 仿真与程序内标量参考值的比较。正数、负数和四组边界值共 6 项已输出通过信息；补充整机测试未完成。

1. 测试环境与方法

1.1 测试环境

项目	配置
处理器	香山昆明湖 V2，`MinimalConfig`，单核
向量宽度	`VLEN=128`
点积通路	复用 `VFEX0/VIMAC`，单微操作两周期，输出端组合归约
仿真器	按当前源码重建的 `XiangShan/build/emu`，保留波形支持，本轮不生成波形
测试应用	`nexus-am/apps/vdot-csr`，`ARCH=riscv64-xs BASIC=1`
编译工具	GCC 9.4.0、Clang 19.1.7、Verilator 5.052
编译方式	C 使用 `rv64gc` 标量指令；汇编使用 `rv64gcv`
检查方式	C 标量计算参考值，比较完整 128 位目标及相关向量 CSR

硬件基于 XiangShan 提交 `16ae9ddcda54fc9a2fddffad73174cf793ac7814` 和 yunsuan 提交 `cadd3c2f43096e253d61296b33ac697be8354e29`，实现改动见源码补丁。

main.c 准备输入和参考值，vdot.S 加载源寄存器组、设置向量配置、执行点积并保存结果。汇编通过 `.word` 表示自定义编码。

1.2 编译与运行

```
cd /home/xpg/cie/xs-env
source env.sh
export JAVA_HOME=/opt/graalvm-jdk-21
export PATH="$JAVA_HOME/bin:/usr/local/llvm/bin:$PATH"
export VDOT_LOG_DIR="$XS_PROJECT_ROOT/vdot-docs/logs"
mkdir -p "$VDOT_LOG_DIR"

make -C "$AM_HOME/apps/vdot-csr" ARCH=riscv64-xs BASIC=1 disasm \
  > "$VDOT_LOG_DIR/vdot-basic-app-build.log" 2>&1

cd "$NOOP_HOME"
export VDOT_LOG="$VDOT_LOG_DIR/vdot-basic-rtl.log"
./build/emu -i "$AM_HOME/apps/vdot-csr/build/vdot-basic-riscv64-xs.bin" \
  --no-diff --max-cycles=200000 > "$VDOT_LOG" 2>&1
rg 'PASS|FAIL|HIT GOOD TRAP|HIT BAD TRAP' "$VDOT_LOG"
```

以上命令复用已构建的香山仿真器，仅编译和运行基础应用。`BASIC=1` 在 6 项检查完成后直接返回，不进入补充遍历。程序内比较失败会打印 `FAIL` 并退出；完整运行通过时须同时出现 `PASS: vdot basic 6 checks` 和 `HIT GOOD TRAP`。

现有 RTL 测试结果来自此前运行的 212 项程序：前三组通过后，程序在补充阶段被手动停止，具体记录见第 4 节。上述命令用于后续单独复现基础 6 项，本次文档修改未重新执行测试。

1.3 统一校验规则

```
int32_t sum = 0;
for (unsigned i = 0; i < vl; ++i)
    sum += (int32_t)a[i] * (int32_t)b[i];
```

每次普通点积检查低 32 位等于标量结果、高 96 位为零；`vl=0` 时改为检查整个目标与执行前一致。此外检查 `vstart=0`，以及 `vl`、`vtype`、`vxrm`、`vxsat` 保持执行前的设置。

两个源数组各有 128 字节，先用整寄存器加载指令装入源组，再设置被测 `vl` 和 LMUL。未参与计算的元素仍含非零数据，用于检查 `vl` 边界。目标预置非零内容；重叠用例则按目标原本保存的源数据检查零长度保留。

2. 基础功能测试

2.1 正数点积测试

测试代码设计

使用 `SEW=8、LMUL=1、vl=16`，两个源的前 16 项均为 `1…16`。执行后保存整个目标寄存器：

```
for (unsigned i = 0; i < 128; ++i) {
    a[i] = (i % 16) + 1;
    b[i] = (i % 16) + 1;
}
if (run(3, 16, 0, 0, 0)) return 1;
```

`run` 的第一个参数是配置表下标，`3` 对应 `LMUL=1`；其余参数依次为 `vl`、寄存器组合、策略及初始定点 CSR 设置。

```
result = 1×1 + 2×2 + … + 16×16 = 1496
低32位 = 0x000005D8，高96位 = 0
```

测试过程命令

执行第 1.2 节的完整命令，然后查询：

```
rg -F 'PASS: positive INT8 dot' "$VDOT_LOG"
```

测试结果

通过。低 32 位为 `0x000005D8`，高 96 位为零，`vstart=0`；`vl/vtype/vxrm/vxsat` 检查一致。日志输出：

```
PASS: positive INT8 dot, VL=16, result=1496 (0x000005d8)
```

2.2 负数点积测试

测试代码设计

保持 `LMUL=1`、`vl=16`，将第一个源改为 `-1…-16`，第二个源保持 `1…16`，检查负数乘法和负结果补码。

```
for (unsigned i = 0; i < 128; ++i)
    a[i] = -((int)(i % 16) + 1);
if (run(3, 16, 0, 0, 0)) return 1;
```

```
result = (-1)×1 + (-2)×2 + … + (-16)×16 = -1496
低32位 = 0xFFFFFA28，高96位 = 0
```

测试过程命令

```
rg -F 'PASS: negative INT8 dot' "$VDOT_LOG"
```

测试结果

通过。低 32 位为 `0xFFFFFA28`，高 96 位为零，向量 CSR 检查一致。日志输出：

```
PASS: negative INT8 dot, VL=16, result=-1496 (0xfffffa28)
```

2.3 边界值测试

测试代码设计

使用 `LMUL=8`、`vl=128`，每个源占用八个向量寄存器。四组输入覆盖最大值、最小值、异号及零：

```
const int limits[][2] = {
    {127, 127}, {-128, -128}, {-128, 127}, {0, -128}
};
for (unsigned n = 0; n < 4; ++n) {
    for (unsigned i = 0; i < 128; ++i) {
        a[i] = limits[n][0];
        b[i] = limits[n][1];
```

```
    }
    if (run(6, 128, 0, 0, n)) return 1;
}
```

a[i]	b[i]	数学期望值	低 32 位期望值
127	127	2064512	0x001F8080
−128	−128	2097152	0x00200000
−128	127	−2080768	0xFFE04000
0	−128	0	0x00000000

同时检查高 96 位为零。第二、三组分别覆盖当前 VLEN=128、SEW=8、LMUL≤8 范围内的点积最大、最小值。

测试过程命令

```
rg -F 'PASS: boundary values' "$VDOT_LOG"
```

测试结果

四组均通过，完整目标及向量 CSR 与参考值一致。日志输出：

```
PASS: boundary values, LMUL=8, VL=128, 4 checks
```

3. 此前已完成的补充测试

3.1 共享 VIMAC 回归及原有向量加法

测试代码设计

使用 apps/vimac-vdot-regress 的 62 项参考用例，检查原有乘法、乘加、加宽、掩码、尾部保持及定点饱和。再交错执行点积、vmul、vmacc、vsmul，检查六个完整向量及粘滞 vxsat。交错序列的点积按当前 vl=16 计算参考值。另运行原有 apps/vadd。

测试过程命令

```
make -C "$AM_HOME/apps/vimac-vdot-regress" ARCH=riscv64-xs disasm
"$NOOP_HOME/build/emu" \
  -i "$AM_HOME/apps/vimac-vdot-regress/build/vimac-vdot-regress-riscv64-xs.bin" \
  --no-diff --max-cycles=1000000 \
  > "$VDOT_LOG_DIR/vimac-regression.log" 2>&1

make -C "$AM_HOME/apps/vadd" ARCH=riscv64-xs disasm
"$NOOP_HOME/build/emu" -i "$AM_HOME/apps/vadd/build/vadd-riscv64-xs.bin" \
```

```
    --no-diff --max-cycles=1000000 \
    > "$VDOT_LOG_DIR/vadd.log" 2>&1
```

测试结果

全部通过。共享通路日志输出：

```
PASS: VIMAC 62 multiply/accumulate/widen/mask/saturate cases
PASS: interleaved vdot/vmul/vmacc/vsmul, 6 full vectors, sticky vxsat preserved
HIT GOOD TRAP at pc = 0x8000044e
```

`vadd` 输出 `PASS: VL=2, result=4f 52 ff ff`，并在 `pc=0x8000020e` 触发 `HIT GOOD TRAP`。完整日志分别见 VIMAC 回归和 vadd 回归。

3.2 模块级 RTL 测试

`XiangShan/src/test/scala/xiangshan/backend/fu/vector/VDotCsrTest.scala` 包含运算通路和译码拆分两项测试。前者检查全部合法 LMUL、各自全部合法 `vl`、极值、零长度保留、异常配置及逐周期交替输入点积和普通乘法；后者检查微操作数量、源寄存器及临时寄存器连接、目标重叠和非法状态。

```
cd "$NOOP_HOME"
mill -i -Djvm-xmx=16G xiangshan.test.testOnly
xiangshan.backend.fu.vector.VDotCsrTest
```

此前两项均通过，日志见 模块测试日志。测试夹具已适配本机 Verilator 5.052 的宽数据接口。测试文件保留在工作区，不包含在硬件源码补丁中。

4. RTL 测试结果汇总

测试日期：2026-11-08。使用按当前源码构建的香山仿真器，以程序内标量计算为参考。构建日志见 emu-build.log。

测试内容	项数	结果
正数点积，LMUL=1、vl=16	1	通过，结果 1496
负数点积，LMUL=1、vl=16	1	通过，结果 −1496
边界值，LMUL=8、vl=128	4	通过
合计	6	三组均输出 PASS

```
PASS: positive INT8 dot, VL=16, result=1496 (0x000005d8)
PASS: negative INT8 dot, VL=16, result=-1496 (0xfffffa28)
PASS: boundary values, LMUL=8, VL=128, 4 checks
```

原始日志见 RTL 测试日志。当时运行的是包含补充检查的 212 项程序，在上述三组完成后按用户要求停止，未输出完整套件的通过标志。因此，仅确认基础 6 项通过，不将 212 项套件记为全部通过。

第 3 节的共享 VIMAC、向量加法及模块测试已分别完成，结果和日志单独列出。零长度、其他 LMUL、寄存器重叠及非法指令等补充检查尚未完成整机验证。